

模型配置与流式对话完整流程说明

本文档说明本项目中“大模型配置管理 + 前端发起流式请求 + 后端调用大模型 + 后端处理模型返回并流式返回前端”的完整实现流程。

1. 整体链路

浏览器前端

↓ 1. 加载模型列表 GET /api/models

后端模型管理接口

↓ 2. 读取 llm_model_configs 表

SQLite / MySQL / PostgreSQL 数据库

浏览器前端

↓ 3. 发送对话 POST /api/chat/stream, 接收 text/event-stream

后端流式对话接口

↓ 4. 根据 model_id 查询模型配置, 构造模型客户端

BaseLLMClient / OpenAICompatibleLLMClient

↓ 5. 调用内网大模型 /v1/chat/completions

内网大模型服务

↓ 6. 返回 SSE 或 JSON 流式数据

后端 StreamingResponse

↓ 7. 标准化为 data: ... \n\n 返回给前端

浏览器前端逐字追加显示

2. 后台模型配置

2.1 模型表定义

代码位置:

service/app/models.py

新增 ORM 模型:

```
class LLMModelConfig(Base):
    __tablename__ = 'llm_model_configs'

    id: Mapped[int] = mapped_column(primary_key=True, index=True)
    name: Mapped[str] = mapped_column(String(100), nullable=False)
    base_url: Mapped[str] = mapped_column(String(255), nullable=False)
    model_name: Mapped[str] = mapped_column(String(100), nullable=False, index=True)
    api_key: Mapped[str] = mapped_column(String(255), nullable=False)
    provider: Mapped[str] = mapped_column(String(50), nullable=False, default='openai-c
    is_default: Mapped[bool] = mapped_column(Boolean, nullable=False, default=False)
    status: Mapped[str] = mapped_column(String(20), nullable=False, default='enabled')
    created_at: Mapped[datetime] = mapped_column(...)
```

字段说明：

字段	说明
name	页面展示用模型名称
base_url	模型服务地址，例如 http://114.242.210.44:8000/v1
model_name	实际调用时传给模型服务的模型名，例如 jasure-llm
api_key	模型服务密钥，只在后端保存，不返回给前端
provider	模型协议类型，目前为 openai-compatible
is_default	是否默认模型
status	是否启用

2.2 默认模型初始化

代码位置：

```
service/app/crud/model_configs.py
service/app/main.py
```

默认模型配置定义在：

```
DEFAULT_MODEL_CONFIG = schemas.ModelConfigCreate(  
    name='jasure-llm 默认模型',  
    base_url='http://114.242.210.44:8000/v1',  
    model_name='jasure-llm',  
    api_key='sk-ad0eca1e43bc60f825372c496f131e53',  
    provider='openai-compatible',  
    is_default=True,  
    status='enabled',  
)
```

应用启动时执行：

```
def initialize_database() -> None:  
    Base.metadata.create_all(bind=engine)  
    with SessionLocal() as db:  
        crud.ensure_default_model_config(db)
```

作用：

1. 自动创建数据库表。
2. 如果默认模型不存在，则写入默认模型。
3. 如果默认模型已存在，则确保它是启用状态并设为默认模型。

3. 后台模型管理接口

代码位置：

```
service/app/routes/model_configs.py
```

3.1 查询模型列表

接口：

```
GET /api/models
```

实现逻辑：

```
@model_configs_router.get('/models')
def read_models():
    with SessionLocal() as db:
        configs = crud.list_model_configs(db, enabled_only=True)
        payload = [schemas.serialize_model_config(config) for config in configs]
    return api_response(data=payload)
```

注意：序列化方法不会返回 `api_key`，避免密钥泄露到前端。

3.2 查询默认模型

接口：

GET `/api/models/default`

从数据库读取 `is_default=True` 且 `status='enabled'` 的模型。

3.3 新增模型配置

接口：

POST `/api/models`

请求体示例：

```
{
    "name": "本地模型",
    "base_url": "http://127.0.0.1:8000/v1",
    "model_name": "local-llm",
    "api_key": "sk-xxx",
    "provider": "openai-compatible",
    "is_default": false,
    "status": "enabled"
}
```

4. 模型调用基类

代码位置：

service/app/llm/base.py

4.1 基类 BaseLLMClient

```
class BaseLLMClient(ABC):
    def __init__(self, *, base_url: str, model_name: str, api_key: str, timeout: float)
        self.base_url = base_url.rstrip('/')
        self.model_name = model_name
        self.api_key = api_key
        self.timeout = timeout

    @abstractmethod
    def stream_chat(...) -> Iterator[str]:
        raise NotImplementedError
```

作用：

- 抽象出统一模型调用接口。
- 后续如果接入其他厂商模型，只需要新增子类实现 `stream_chat`。
- 上层路由无需关心底层模型服务差异。

4.2 OpenAI 兼容客户端

```
class OpenAICompatibleLLMClient(BaseLLMClient):
    def stream_chat(...) -> Iterator[str]:
        payload = {
            'model': self.model_name,
            'messages': [...],
            'temperature': temperature,
            'stream': True,
        }
```

调用目标：

`{base_url}/chat/completions`

例如默认模型最终请求地址为：

http://114.242.210.44:8000/v1/chat/completions

请求头：

```
headers={
    'Authorization': f'Bearer {self.api_key}',
    'Content-Type': 'application/json',
    'Accept': 'text/event-stream',
}
```

处理逻辑：

1. 向模型服务发起 POST /chat/completions 。
2. 设置 stream=True 。
3. 按行读取模型返回。
4. 如果上游已经返回 data: 开头的 SSE 行，则直接透传。
5. 如果上游只返回 JSON 行，则补齐为标准 SSE：

```
yield f'data: {line}\n\n'
```

5. 后台流式对话接口

代码位置：

service/app/routes/model_configs.py

接口：

```
POST /api/chat/stream
Content-Type: application/json
Accept: text/event-stream
```

请求体示例：

```
{
    "model_id": 1,
    "messages": [
        { "role": "system", "content": "你是一个中文助手。" },
        { "role": "user", "content": "你好，请介绍一下你自己。" }
    ],
    "temperature": 0.7,
    "max_tokens": 1024
}
```

核心实现：

```
@model_configs_router.post('/chat/stream')
def stream_chat(data: dict[str, Any] | None = Body(default=None)):
    # 1. 校验 messages、model_id、temperature、max_tokens
    # 2. 根据 model_id 查询模型配置；未传则使用默认模型
    # 3. 创建 OpenAICompatibleLLMClient
    # 4. 通过 StreamingResponse 返回 SSE
```

模型配置读取：

```
config = crud.get_model_config(db, model_id) if model_id > 0 else crud.get_default_mode
```

创建客户端：

```
client = OpenAICompatibleLLMClient(
    base_url=config.base_url,
    model_name=config.model_name,
    api_key=config.api_key,
)
```

流式返回：

```
def event_generator():
    try:
        for chunk in client.stream_chat(...):
            yield chunk
    except LLMClientError as exc:
        yield _sse_error(str(exc), model_name=model_name)
        yield 'data: [DONE]\n\n'
```

响应对象：

```
return StreamingResponse(
    event_generator(),
    media_type='text/event-stream',
    headers={
        'Cache-Control': 'no-cache',
        'X-Accel-Buffering': 'no',
    },
)
```

6. 后台返回给前端的数据结构

后端统一返回标准 SSE 数据流。

正常片段示例：

```
data: {"id":"chatcmpl-xxx","object":"chat.completion.chunk","created":1710000000,"model
```

结束片段示例：

```
data: [DONE]
```

异常片段示例：

```
data: {"id":"chatcmpl-error-xxx","object":"chat.completion.chunk","created":1710000000,
```

前端重点读取：


```
chunk?.choices?.[0]?.delta?.content
```

如果存在：

```
chunk?.error?.message
```

则作为错误消息展示。

7. 前端模型列表加载

代码位置：

```
web/src/services/api.ts  
web/src/components/ModelTableCard.tsx
```

7.1 API 封装

```
export async function fetchModels() {  
  return request<ModelConfigRecord[]>({  
    method: 'GET',  
    url: '/models',  
  })  
}
```

7.2 页面加载模型

```
const loadModels = useCallback(async () => {  
  const res = await fetchModels()  
  const result = unwrapList<ModelConfigRecord>(res)  
  const nextModels = result?.items ?? []  
  setModels(nextModels)  
  setSelectedModelId(  
    current => current ?? nextModels.find(item => item.is_default)?.id ?? nextModels[0]  
  )  
}, [])
```

作用：

1. 页面进入“模型”页时加载模型列表。
2. 默认选中 `is_default=true` 的模型。
3. 如果没有默认模型，则选中第一条模型。

8. 前端发起流式请求

代码位置：

```
web/src/services/api.ts
```

核心函数：

```

export async function streamChat(payload, handlers) {
  const response = await fetch(`${getApiBaseUrl()}/chat/stream`, {
    method: 'POST',
    headers: {
      Accept: 'text/event-stream',
      'Content-Type': 'application/json',
    },
    body: JSON.stringify(payload),
    signal: handlers.signal,
  })

  const reader = response.body.getReader()
  const decoder = new TextDecoder('utf-8')
  let buffer = ''

  while (true) {
    const { value, done } = await reader.read()
    if (done) break
    buffer += decoder.decode(value, { stream: true })
    const events = buffer.split('\n\n')
    buffer = events.pop() ?? ''

    for (const event of events) {
      const line = event
        .split('\n')
        .find(item => item.startsWith('data:'))
        ?.replace(/^data:\s?/, '')

      if (!line || line === '[DONE]') continue

      const chunk = JSON.parse(line)
      const content = chunk?.choices?.[0]?.delta?.content
      if (content) handlers.onDelta(String(content))
    }
  }
}

```

这里没有使用 axios，因为 axios 默认更适合普通 JSON 请求，而 fetch + ReadableStream 更适合浏览器端处理流式响应。

9. 前端页面对话逻辑

代码位置：

web/src/components/ModelTableCard.tsx

用户点击发送后：

```
await streamChat(
  {
    model_id: selectedModelId,
    messages: [
      { role: 'system', content: DEFAULT_SYSTEM_PROMPT },
      ...nextMessages.map(({ role, content }) => ({ role, content })),
    ],
    temperature: 0.7,
  },
  {
    signal: controller.signal,
    onDelta: (delta) => {
      setMessages((current) =>
        current.map((item) =>
          item.id === assistantMessage.id
            ? { ...item, content: item.content + delta }
            : item,
        ),
      )
    },
  },
)
```

前端处理过程：

1. 用户输入问题。
2. 前端将用户消息加入消息列表。
3. 前端先创建一条空的 assistant 消息。
4. 调用 /api/chat/stream 。
5. 每收到一个 delta.content ，就追加到 assistant 消息中。
6. 页面呈现逐字输出效果。

10. 停止生成

代码位置：

```
web/src/components/ModelTableCard.tsx
```

前端通过 AbortController 中断请求：

```
const controller = new AbortController()
abortRef.current = controller
```

停止按钮逻辑：

```
const stopStreaming = useCallback(() => {
  abortRef.current?.abort()
  abortRef.current = null
  setStreaming(false)
}, [])
```

说明：

- 前端中断浏览器请求。
- 后端连接通常会随客户端断开而结束。
- 具体模型服务是否立即停止生成，取决于上游模型服务实现。

11. 当前涉及的主要文件

文件	作用
service/app/models.py	定义 llm_model_configs 模型配置表
service/app/crud/model_configs.py	模型配置 CRUD 和默认模型初始化
service/app/schemas/model_configs.py	模型配置序列化，隐藏 api_key
service/app/llm/base.py	模型调用基类和 OpenAI 兼容客户端
service/app/routes/model_configs.py	模型管理接口和流式对话接口
service/app/routes/__init__.py	注册模型相关路由

文件	作用
service/app/main.py	启动时创建表并写入默认模型
web/src/services/api.ts	前端模型接口和流式请求封装
web/src/components/ModelTableCard.tsx	模型选择和对话页面
web/src/App.tsx	模型 Tab 页面接入
web/src/App.css	对话窗口样式

12. 调试建议

12.1 启动后端

```
cd service
uvicorn app.main:app --host 127.0.0.1 --port 8090 --reload
```

12.2 启动前端

```
cd web
npm run dev
```

12.3 检查模型列表

```
curl http://127.0.0.1:8090/api/models
```

12.4 检查流式接口

```
curl -N \  
  -H "Content-Type: application/json" \  
  -H "Accept: text/event-stream" \  
  -X POST http://127.0.0.1:8090/api/chat/stream \  
  -d '{  
    "messages": [  
      {"role": "user", "content": "你好"}  
    ]  
  }'
```

如果能看到多段 `data: ...` 输出，说明后端流式链路正常。

13. 扩展其他模型服务

如果后续需要支持非 OpenAI 兼容协议，可以：

1. 在 `service/app/llm/base.py` 中新增客户端类，例如 `QwenLLMClient`、`ClaudeLLMClient`。
2. 继承 `BaseLLMClient`。
3. 实现统一的 `stream_chat` 方法。
4. 在路由中根据 `provider` 字段选择不同客户端。

示例：

```
if config.provider == 'openai-compatible':  
    client = OpenAICompatibleLLMClient(...)  
elif config.provider == 'custom-provider':  
    client = CustomProviderLLMClient(...)  
else:  
    raise HTTPException(status_code=400, detail='不支持的模型服务类型。')
```

这样前端无需改动，仍然只调用 `/api/chat/stream`。